Opening the Box

Reading a language model's internals

Twenty lectures of black boxes

In chapter 5 we learned to explain a model without ever looking inside it.

- SHAP splits the prediction between the input features.
- Permutation importance shuffles a column and watches the score fall.
- Partial dependence sweeps one feature and plots the response.

> Every one of those methods treats the model as a **function you can only query**. Feed it inputs, read the outputs, infer what mattered. None of them ever reads a weight.

That was not a limitation of the methods. It was a decision - and for a gradient-boosted tree it is still the right one.

Today we make the other decision.

A sentence a small model gets right

> *"Then, Alex and William went to the restaurant. Alex gave a book to ___"*

GPT-2 small - 124 million parameters, released in 2019 - answers **" William"**.

You do it instantly, and if asked how, you would say something like: *two names are introduced, one of them comes back as the giver, so the other one is the receiver.*

That is an **algorithm**. The question for this chapter is whether the model runs anything like it - and how we could possibly find out.

Predict first

We have complete access to this model. Every one of its 124 million numbers is sitting on the disk in front of us, and we can read all of them.

Is knowing every weight the same as understanding the model?

Predict first

We have complete access to this model. Every one of its 124 million numbers is sitting on the disk in front of us, and we can read all of them.

Is knowing every weight the same as understanding the model?

No - and the reason is worth stating precisely.

We have the weights the way you have a wiring diagram of a city's power grid: complete, correct, and no help at all in answering *"why did that street go dark?"* **Access is not understanding.** The gap between them is the entire field.

What mechanistic interpretability is

Using a model's **internals** - its weights and activations - to understand what it is doing.

Note what that definition does **not** say. It does not promise a complete reverse-engineering of the network, and the field has become deliberately pragmatic about this: the goal is useful understanding, not a full circuit diagram of a trillion-parameter model.

The through-line of these three lectures: it is easy to form a story about what is happening inside a network, and hard to show that the story is true. Most of what we teach is how to tell those two apart.

What this does not replace

	Black box (chapter 5)	**White box** (this chapter)
Works on	**any** model - boosted trees, a support vector machine, an API you can only call	neural networks whose **weights you have**
Needs	predictions	weights, activations, and a framework to hook them
Answers	which **input** moved the output	what the **model did**
Cost	seconds	the circuit in lecture 2 took a research team months

Most of you will deploy gradient boosting. For that model SHAP is not the weaker tool - it is the **only** tool. This chapter is not an upgrade over chapter 5. It is a different instrument, for a different object.

Why a small, old model

We will spend three lectures on GPT-2 small: **12 layers**, **12 heads per layer**, **144 heads** in total, and a residual stream of width **768**. It is seven years old.

Why it is the right choice

- every interpretability tool targets it;
- it is the only model where one task has been reverse-engineered completely;
- it runs on a laptop, so **every figure in these lectures is one you can reproduce**.

What transfers, and what does not

- ✓ the residual-stream picture;
- ✓ the causal methods, all of them;
- ✓ the standards of evidence;
- ✗ the **specific circuits**. What we find is a fact about GPT-2, not about every model.

The field has largely **retired** toy-model interpretability as a research direction, for the reason in the right-hand column. It is still the only sane place to *learn* it. Training ground, not frontier.

Outline

The residual stream

Before any technique: what is the thing we are reading?

You have already been told the key fact

In lecture 25 one frame was titled *"the residual stream is the real object"*. It is time to collect on it.

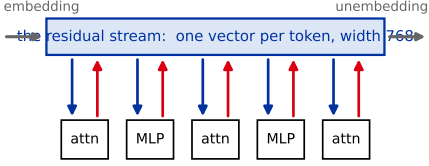
> Every attention head and every multi-layer perceptron (MLP) block **reads** from the stream and **adds** its output back. No component talks to any other directly. The stream is the only channel in the building.

Two claims are hiding in that box: that each block *adds* rather than overwrites, and that the stream is the *only* route between blocks. Both are checkable.

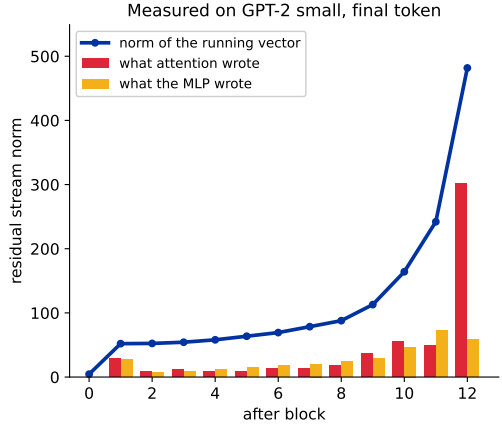
The stream, and what it looks like in a real model

No component talks to any other directly.

The stream is the only channel in the building.



nothing is ever overwritten



The final vector is exactly the embedding plus all 24 writes (max error 0e+00).

The right panel is the claim, tested: we rebuilt the final vector from the embedding plus all 24 writes and compared it to the real one. They agree to **zero error**.

Everything is a sum

Nothing overwrites the stream. Each component **adds**. So the vector that arrives at the output is nothing more than:

$$\underbrace{\text{final residual}}_{\text{768 numbers}} = \underbrace{\text{embedding}}_{\text{the tokens}} + \underbrace{\sum h_i}_{\substack{\text{144 heads} \\ \text{what each head wrote}}} + \underbrace{\sum m_j}_{\substack{\text{12 MLPs} \\ \text{what each MLP wrote}}}$$

That is the entire architecture, from the point of view of this chapter. One long sum, with **157 terms** - and they are not small. On our sentence the stream leaves the embedding with norm **4.5** and reaches the unembedding at **482**: the blocks write about a hundred times more into it than the tokens did.

Hold that thought. First we need something to measure.

The metric: logit difference

Our sentence has exactly two plausible answers and only one is right. So compare them directly:

$$\text{logit difference} = \underbrace{\text{logit}(\text{“ William”})}_{\text{correct}} - \underbrace{\text{logit}(\text{“ Alex”})}_{\text{the other name}}$$

Why not accuracy?

A step function. It cannot tell you a change made the model *slightly* less sure - and most changes do exactly that.

Why not loss?

Loss mixes in all 50,257 tokens. We want the one comparison the task is about, and nothing else.

One sentence, by hand

> *"Then, Alex and William went to the restaurant. Alex gave a book to . . . "*

The model's top five predictions for the next token, with their logits:

token	logit
" William"	**17.96**
" the"	15.50
" his"	14.79
" them"	13.96
" a"	13.69
" Alex"	12.62

$$\text{logit difference} = 17.96 - 12.62 = \mathbf{5.34}$$

Across 64 such sentences the average is $+\mathbf{3.60}$; we always quote the average.

So the logit difference is a sum too

The step from the residual stream to a logit is a **matrix multiply** - the unembedding. Matrix multiplication is linear, and linear means it distributes over a sum:

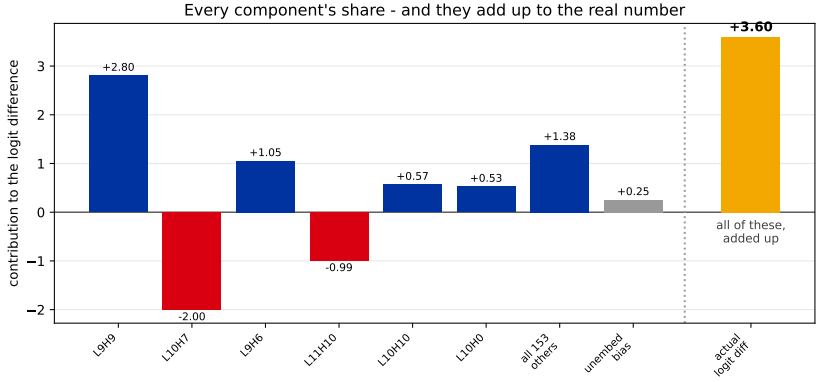
$$W_U(a+b+c) \;=\; W_U a \;+\; W_U b \;+\; W_U c$$

Apply that to the 157-term sum from two slides ago, and the logit difference splits the same way:

$$\text{logit difference} \;=\; \underbrace{c_{\text{embed}} + c_{\text{head 0.0}} + \cdots + c_{\text{head 11.11}} + \cdots}_{\text{one number per component}}$$

Each component gets a **single number**: how much it pushed toward " William" over " Alex". That number is the component's **direct logit attribution**.

Do not take my word for it



Six largest contributors, then everything else lumped together, then GPT-2's unembedding bias - a fixed offset the model does not compute. Orange is the logit difference from an ordinary forward pass.

The sum is exact

159 component contributions, added up	+3.3439
unembedding bias (a constant, not computed by the model)	+0.2520
total	**+3.5960**
logit difference from an ordinary forward pass	+3.5960
difference	**0.00000**

This is why per-component attribution exists at all. "How much did head 9.9 contribute?" has an exact answer, because the architecture really is a sum. Remove the residual connections and most of this chapter stops working.

One honest caveat: LayerNorm sits between the stream and the unembedding and is *not* linear. We hold its scaling factor fixed at the value the real forward pass produced - which is what makes the table above come out exact.

The one piece of code in this lecture

```
logits, cache = model.run_with_cache("Then, Alex and William went to the...")
cache["resid_post", 9]     # the stream after layer 9
cache["pattern", 9]        # attention weights, all 12 heads
cache["z", 9]           # what each head wrote
```

One forward pass, and every intermediate value is kept instead of thrown away. That is the whole trick, and it is why libraries like TransformerLens exist: not new mathematics, just refusing to discard the activations.

Most of the practical difficulty in this field is **careful indexing** - which layer, which position, which head. Not the concepts. Budget your frustration accordingly.

The logit lens

If the answer is built gradually, can we watch it happen?

One idea, one sentence

Take the residual stream **halfway through** the model, apply the unembedding that is normally used only at the end, and read off what the model **would have said** if it had stopped there.

Nothing about this is principled - we are applying the final layer's decoder to a vector it was never trained to decode. It works well enough on GPT-2 to be useful, which is a sentence that should already make you slightly suspicious. We will return to that.

Introduced by nostalgebraist (2020), as a blog post rather than a paper - which is common in this field.

Predict first

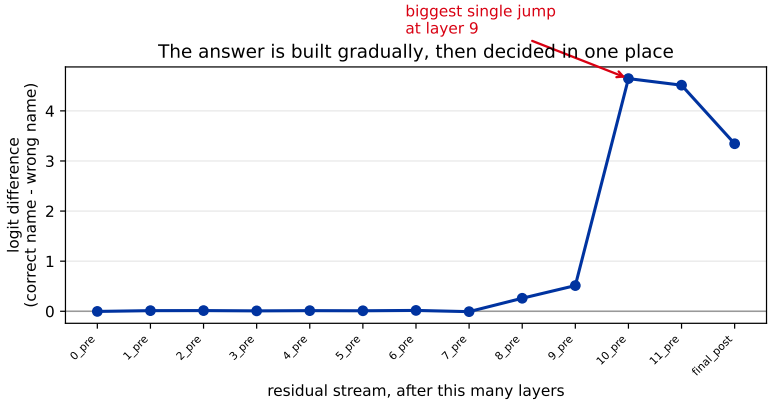
We are about to run the logit lens at every layer of GPT-2 small and plot the logit difference as it develops.

Sketch the shape of that curve before you see it.

Does the answer build up **steadily** across the twelve layers,
or does it arrive **somewhere in particular**?

Most people draw a diagonal line. Very few draw what actually happens.

Watching the answer arrive



- **Eight layers** of essentially nothing.
- Entering **layer 9**, it jumps.
- Then a slow drift to the final value.

Logit difference read off the stream after each layer, averaged over 64 sentences.

One number to carry into next week

The decision is **not** spread evenly through the network. Something specific happens at **layer 9**, and whatever it is, it is responsible for most of the answer.

That is our first real hypothesis, and it is narrow enough to test: **look at layer 9**.

Remember the number. Next lecture we go looking for the components that do this work, using a completely different method that knows nothing about this graph.

They will be in **layer 9**.

Before you trust it

The logit lens is **not** a window into "what the model is thinking". It is a decoder being used out of distribution.

- It works reasonably on GPT-2 and **poorly on many other models** - representations drift between layers, and the final unembedding does not match them.
- The fix people use - training a small decoder per layer, the "tuned lens" - exists precisely because the plain version is unreliable.
- It shows what is **linearly readable**, which is not the same as what is **there**, which is not the same as what is **used**.

That last distinction is important enough that the next section is about nothing else.

Probes

Asking the model a yes/no question about its own activations

A probe is logistic regression

You already know how to build one. From lecture 11:

Lecture 11	**Here**
design matrix of features	the residual stream, 768 numbers per token
label: did the customer subscribe?	label: is one of these names repeated?
fit logistic regression	fit logistic regression

Train a classifier on activations. If it succeeds, that information is present in the stream in a **linearly readable** form. That is the whole method.

We train one probe per layer, cross-validated, on 128 sentences balanced 50/50 between "a name repeats" and "three different names".

Predict first

We train one probe per layer, asking only: *is one of these names repeated?*

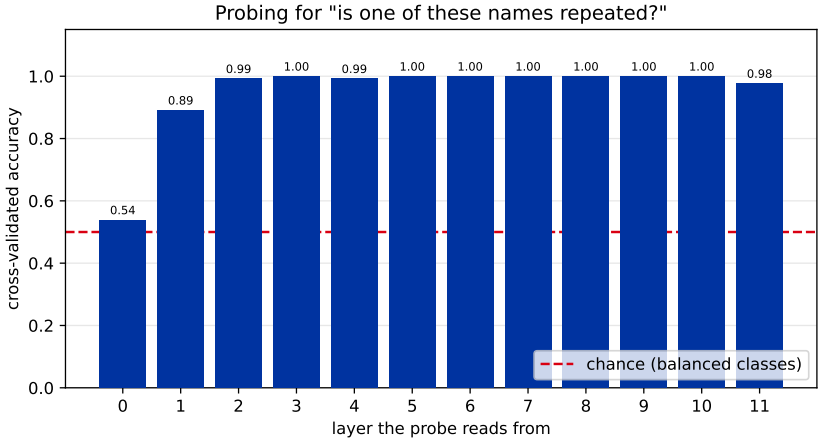
At which layer do you think the probe first hits 100%?

Late, near layer 9, where we just saw the answer appear?

Or somewhere else entirely?

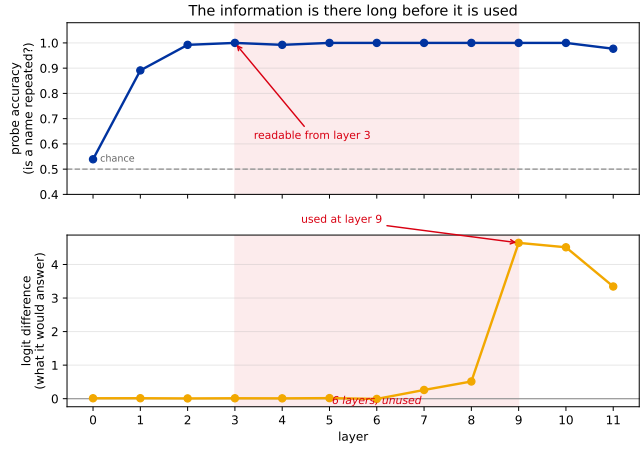
Commit to a layer number before the next slide.

The information shows up early



Five-fold cross-validated accuracy, one probe per layer, reading the final token position. **100% from layer 3** - six layers before the model does anything with it.

The two graphs, together



Same x axis. Top: what is **readable**. Bottom: what is **used**.

Six layers of knowing and not acting

The information sat in the residual stream, perfectly readable, from layer 3 to layer 9 - and the model's answer did not move.

A probe tells you a fact is **present**. It does not tell you the model **uses** it.

You can read every ingredient off a label without knowing which ones the recipe calls for. The probe reads the label. It has nothing to say about the cooking.

This gap is the single most useful thing in the lecture, because it is the mistake people actually make: finding information with a probe and reporting it as what the model is *doing*.

How a probe can fool you

- **Presence is not use** - the six-layer gap above, measured on our own data.
- **The probe may be doing the work.** A powerful enough probe can find structure the model never exploits. Keep probes linear, and report the chance baseline.
- **Correlated features.** The probe may be reading something that merely travels with what you asked about.

Our probe hitting **100%** by layer 3 should make you ask whether the task was too easy, not congratulate the model. An easy task is fine here, because the point of the figure is the **contrast** with the logit lens - not the accuracy.

Where probes actually earn their keep

Othello-GPT. Train a transformer only on sequences of legal Othello moves - it never sees a board. Then probe its activations for the state of each square.

The probes work. The model built an internal **board** it was never given.

> This is the strongest thing probes have shown: models can construct genuine internal **world models** from data that only implies them.

And the practical version: linear probes on activations are **deployed today** for content monitoring in real systems. Of everything in these three lectures, this is the technique most likely to be useful in your job - it is cheap to train, cheap to run, and it works.

Li et al. (2022); the linear-probe follow-up is Nanda et al. (2023).

Reading attention heads

144 of them. Which ones are doing the work?

What a head does, restated

From lecture 24: a head computes queries, keys and values, and produces a weighted sum of values.

For our purposes, one sentence is enough:

> An attention head **moves information from one position to another**. The attention pattern says *where from*; what it writes is a separate question entirely.

That split is the whole content of this section. The pattern is easy to look at, easy to screenshot, and easy to over-read.

Where it looks is not what it writes

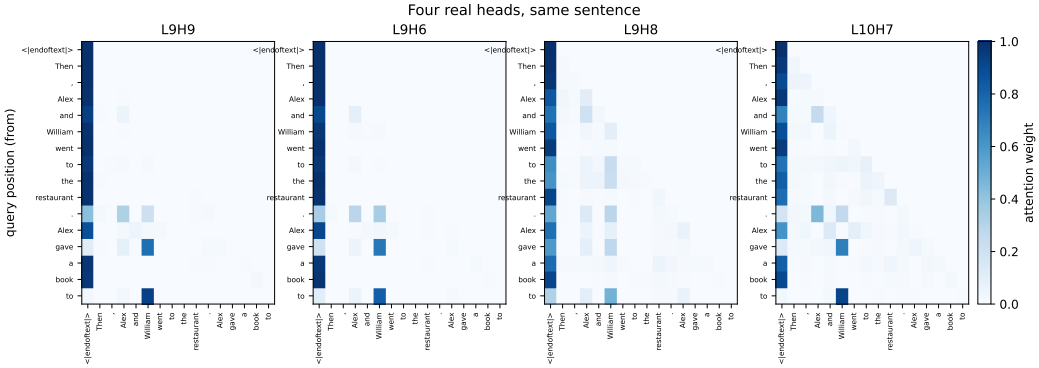
A head does two separable things, and we can only see one of them:

	Where it looks	**What it writes**
Set by	queries and keys	values, and the output matrix
Shown by	the attention pattern	nothing you can look at
Easy to plot?	yes - it is a heatmap	no - it is a 768-dimensional vector

Every attention picture you have ever seen - including the four on the last slide - shows only the **left** column. A head can stare straight at the right answer and write a vector that no later component reads.

We have no way to check that yet. That is what the next lecture is for.

Four real heads on our sentence



Rows are query positions ("from"), columns are key positions ("to"). The dark first column is every head parking attention on the first token - an **attention sink**, and mostly not about the sentence.

Induction heads

The most famous named circuit in the field, and it is easy to state:

$$\ldots\ \mathbf{A}\ \mathbf{B}\ \ldots\ \mathbf{A} \rightarrow \textit{predict}\ \mathbf{B}$$

"This pattern appeared before. What came next last time?"

Test it with a sequence of **random** tokens repeated twice. There is nothing to learn from the language - the only way to predict the second half is to look back at the first.

This is a large part of how in-context learning works. It is also why a model can pick up a pattern you invent inside a single prompt.

Olsson et al. (2022).

Walk through it, one position at a time

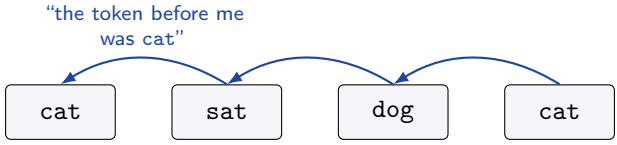
A concrete sequence. The model has seen the first four tokens and must predict the fifth:

The answer a human gives instantly: `sat`. The pattern `cat` appeared before, and `sat` followed it.

The model has no rule for this. What it has is two heads, in two different layers, and neither of them can do it alone.

Step 1 - the previous-token head

In an **early** layer, one head does something that looks pointless: at every position, it attends to the position *before* it, and copies that token's identity into the stream.



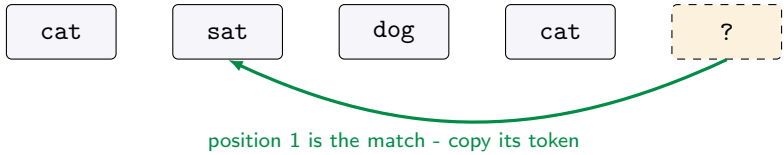
So position 1 now carries a **mark**: *I am preceded by* `cat`.

On its own this is useless. It is bookkeeping done in advance, by a head that has no idea what it will be used for.

Step 2 - the induction head

In a **later** layer, a second head sits at the final position, whose token is `cat`, and asks the stream a question:

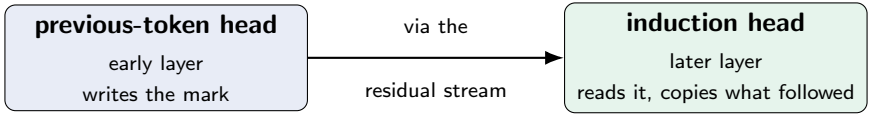
"Which earlier position is marked as being preceded by `cat`?"



Position 1 is marked. Its token is `sat`. The head copies `sat` to the output.

The mark was written by a head that did not know why. It was read by a head that never saw the original `cat`. **Neither one contains the algorithm.**

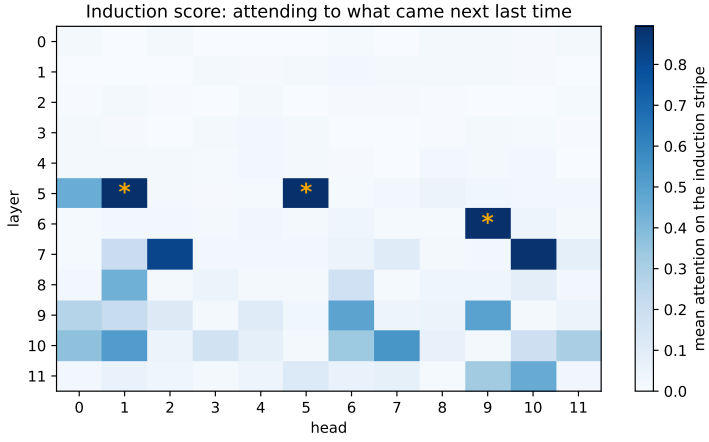
That is what "circuit" means



A **circuit** is a behaviour that lives in the *composition* of components, not in any one of them. Delete either head and the behaviour disappears - but you will never find it by examining either head alone.

This is why "which neuron represents X" is so often the wrong question, and it is the shape of every result in the next lecture.

Finding them, in one pass



Stars mark the top three: **L5H5** (0.89), **L6H9** (0.89), **L5H1** (0.89). Layers 5 and 6 - the middle of the model.

Predict first

We have found a head whose attention pattern is exactly what an induction head should look like.

Does that prove the head is **doing** induction?

Predict first

We have found a head whose attention pattern is exactly what an induction head should look like.

Does that prove the head is **doing** induction?

No.

A head can attend somewhere and write **nothing useful** when it gets there. Attention says where a head *looked*. It says nothing about what it *did*, or whether anything downstream **read** the result.

Next lecture opens with three heads in the same layer, all looking at the right answer. One of them contributes exactly nothing, and the pictures will not tell you which.

Naming what you find

The most natural thing to do, and the easiest way to be wrong

Max-activating examples

The standard way to name a neuron or a head: run a lot of text through, keep the inputs that make it fire hardest, and look at them.

Top activations for a neuron:

```
‘‘...the Golden Gate Bridge ...’’
‘‘...crossing the bridge at dawn ...’’
‘‘...San Francisco’s famous span ...’’
```

It is genuinely useful, it is how most features get named, and it has a failure mode serious enough to have its own name.

Most neurons are not about one thing

Look at enough GPT-2 neurons and the pattern is unmistakable: a single neuron fires on several **unrelated** concepts.

> **Polysemanticity.** One neuron, many meanings. It is the normal case, not the exception - which means "find the neuron for X" is usually the wrong question.

You have met the reason already: in the autoencoder chapter, a network tracking 40 concepts in 32 dimensions has no choice but to share directions. We return to it properly in lecture 3, where it stops being a nuisance and becomes the organising idea.

The interpretability illusion

Bolukbasi et al. took a neuron in BERT, collected its top-activating examples on one dataset, and got a clean, convincing story.

Then they ran a **different** dataset through the same neuron and got a different clean, convincing story.

> Both stories were supported by the evidence. At most one of them was right. **Cherry-picking is not a mistake you might make here - it is the default behaviour of the method.**

The defence is dull and non-negotiable: report **randomly selected** examples alongside the top ones, and check across datasets.

Recap

Technique	What it told us	What it cannot tell us
Residual stream	everything is a sum, so attribution is possible	–
Logit lens	the answer appears at layer 9	whether layer 9 *causes* it
Probes	duplication is readable from layer 3	whether the model uses it
Attention patterns	heads look at the right names	whether looking mattered
Max-activating examples	a name for a component	whether the name is correct

Read the right-hand column again. **Every technique in this lecture is correlational.** We have four ways to form a hypothesis and not one way to test one.

Next

Next: intervention.

Instead of watching the model run, we **break** it on purpose - delete a head, swap an activation, and measure what changes. That is the difference between noticing that something looks important and showing that it **is**.

We open with three heads that all look right. One of them does nothing.

Optional viewing before next time: the Welch Labs video on grokking, linked on the chapter page - a complete circuit, in a model small enough to understand entirely.